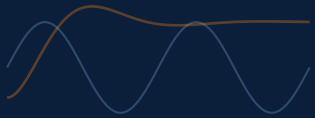
WEEK 1 · DAY 5 / 5

Mechatronics and Robotics Engineering (50 h)



Servo Systems II: FOC, Cascaded Loops, Tuning and Trajectories

3.3 Field-oriented control ▪ 3.4 Cascaded loops ▪ 3.5 Loop tuning & bandwidth ▪ 3.6 Trajectories, feedforward & disturbance compensation

Mechatronics and Robotics Engineering Programme

Week 1 · Day 5 ▪ 2 weeks · 10 days · 50 hours

Agenda and learning outcomes

Code	Session	Net time
3.3	Field-oriented control (Clarke/Park intuition, d–q currents)	1.5 h
3.4	Cascaded current, velocity and position loops	1.5 h
3.5	Loop tuning and bandwidth (inner → outer, inertia ratio, delays)	1.0 h
3.6	Trajectory generation, feedforward, disturbance compensation	1.0 h
	Total (+ 10-min breaks about every hour)	**5.0 h**

One reference joint all day: PM servo motor $p = 4$, $R = 0.5\,\Omega$, $L = 3\,\text{mH}$, $\psi_{\text{PM}} = 0.12\,\text{Vs}$, $k_t = 0.72\,\text{Nm/A}$, $J_m = 3 \times 10^{-4}\,\text{kg m}^2$ (load $3\times$), $V_{\text{dc}} = 325\,\text{V}$, $\hat{I}_{\text{max}} = 20\,\text{A}$, PWM 16 kHz; gear $N = 100$.

By the end of today you can . . .

- explain why FOC turns AC currents into two DC numbers
- sketch the $abc \to \alpha\beta \to dq$ chain and the FOC loop
- calculate PI gains for the current and velocity loops
- explain bandwidth, inertia ratio and the phase cost of delays
- plan a trapezoidal joint move and add feedforward and gravity compensation

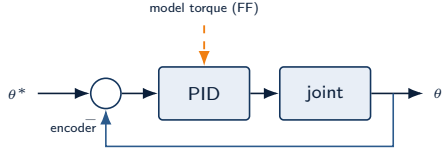
What we learned so far (1/2): control tools from W1D3–W1D4

Feedback and its limits

- PID: P = now, I = past, D = trend of the error (2.5).
- Bode plot and phase margin = distance from oscillation (2.4, 2.6).
- Filters cost phase; notch removes one frequency (2.7).
- A digital loop adds delay of about 1.5 samples (2.8).

Robot manipulator control (2.9)

- Independent-joint control: one servo loop per joint.
- Gravity compensation and computed torque: **predict** the torque instead of waiting for the error.



What we learned so far (2/2): the servo drive, and today's chain

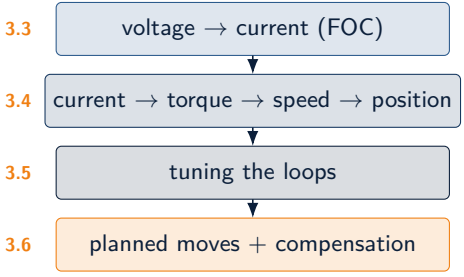
Servo architecture and power stage (3.1 and 3.2)

- Drive modes: torque, velocity, position.
- DC bus → three-leg inverter → PWM sets an **average** voltage on each phase.
- Limits: peak current, bus voltage; braking energy goes back to the DC bus.



Servo drives. Tero Kontkanen, CC BY-SA 3.0, via Wikimedia Commons

Today: from switch commands to a precise joint move



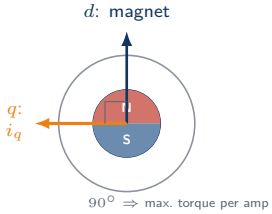
Session 1

3.3 Field-Oriented Control (FOC)

Why field-oriented control? The DC-motor analogy

- Brushed DC motor: the commutator keeps the current at 90° to the magnet $\Rightarrow$ simply $T = k_t\, i$.
- PM synchronous motor (**PMSM**): the magnet is on the rotor and turns – the current must turn with it.
- Analogy: to open a door with least effort, push **perpendicular** to the door – even while it swings.
- **FOC** = electronic commutator: measure the rotor angle θ_e and split the current into
  - i_d – along the magnet (no torque),
  - i_q – 90° ahead (makes torque).

[Blaschke, 1972]; [Leonhard, 2001]; [Krishnan, 2010]

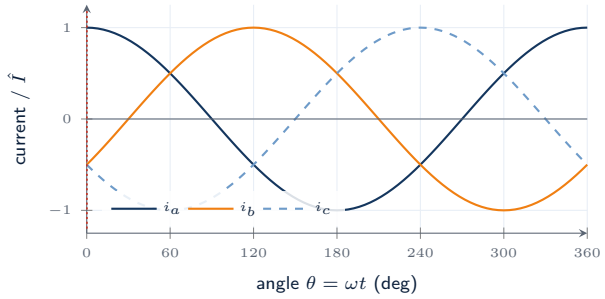




Opened brushless PM motor. Sebastian Koppehel, CC BY 3.0, via Wikimedia Commons

Almost every robot joint uses a PMSM driven by FOC.

Refresher: three-phase sinusoids, 120° apart



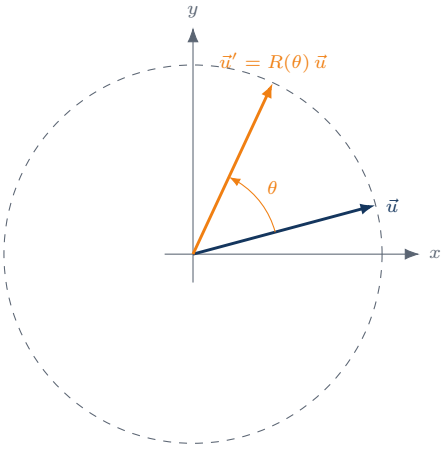
At $\theta = 0$: $i_a = 1$, $i_b = i_c = -0.5 \Rightarrow$ sum $= 0$. True at *every* instant.

$$i_a = \hat{I}\cos\theta$$
$$i_b = \hat{I}\cos(\theta - 120^\circ)$$
$$i_c = \hat{I}\cos(\theta + 120^\circ)$$

- Same amplitude, shifted by one third of a period.
- Sum is zero $\Rightarrow$ only **two** are independent.
- Coils 120° apart in space, fed 120° apart in time $\Rightarrow$ one field arrow of constant length that **rotates** at ω.

Take-away: three wiggling numbers = one rotating arrow.

Refresher: rotating vectors and rotation matrices



Length stays the same, only the direction changes.

- A vector (x, y) = the complex number $x + jy$. Multiplying by $e^{j\theta}$ **rotates** it by θ.
- The same in matrix form:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Example

Rotate $(1, 0)$ by $90°$: $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

Turning the *axes* by $+\theta$ instead = turning the vector by $-\theta$: $(3, 4)$ seen from axes turned $90°$ reads $(4, -3)$.

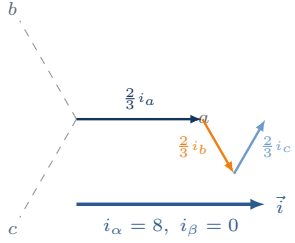
You will meet $R(\theta)$ again in W2D1 (robot frames).
Take-away: $R(-\theta)$ = "what the vector looks like to a turned observer".

Clarke transform: three phases $\rightarrow$ two axes ($\alpha\beta$)

Clarke transform (axes fixed to the stator)

$$\begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} i_a \\ i_b \\ i_c \end{bmatrix}$$

- Each phase current pushes along its own coil axis; add the three pushes as arrows, write the result in x–y form.
- Because $i_a + i_b + i_c = 0$: $i_\alpha = i_a$, $i_\beta = (i_a + 2i_b)/\sqrt{3}$ – two current sensors suffice.
- Factor $\frac{2}{3}$: arrow length = peak phase current $\hat{I}$.

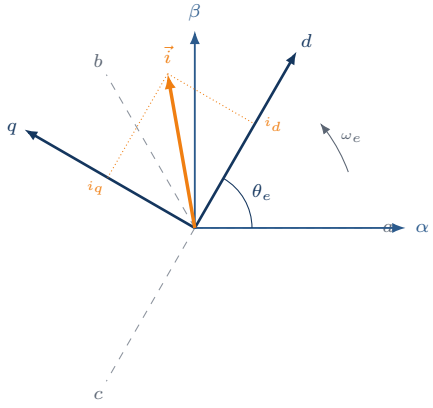


Example

$i_a = 8\,\text{A}$, $i_b = i_c = -4\,\text{A}$:
$i_\alpha = \frac{2}{3}(8 + 2 + 2) = 8\,\text{A}$,
$i_\beta = (8 - 8)/\sqrt{3} = 0\,\text{A}$.

The arrow still **rotates** at ω_e: i_α, i_β are still sines [Clarke, 1943].

Park transform: looking at the rotor from a rotating camera



d axis glued to the magnet's north pole.

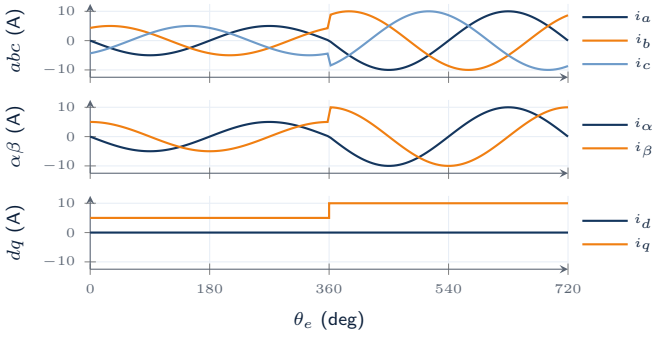
- Imagine a **camera bolted to the rotor**. In its picture the magnet stands still.
- The current arrow rotates at the same speed ⇒ it *also* looks still: two constant numbers i_d, i_q.
- Maths: rotate the axes by θ_e = rotate the vector by $-\theta_e$:

$$\begin{bmatrix} i_d \\ i_q \end{bmatrix} = \begin{bmatrix} \cos\theta_e & \sin\theta_e \\ -\sin\theta_e & \cos\theta_e \end{bmatrix} \begin{bmatrix} i_\alpha \\ i_\beta \end{bmatrix}$$

Take-away: same physical arrow, new coordinates – AC becomes DC.

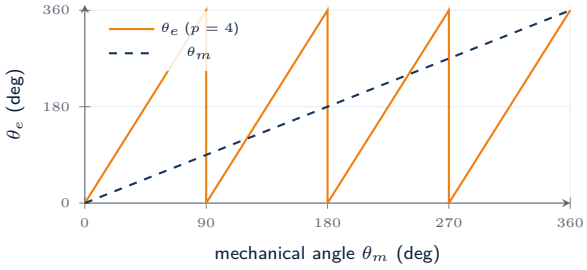
Going further: the $\frac{2}{3}$ (amplitude-invariant) scaling makes power $p = \frac{3}{2}(v_d i_d + v_q i_q)$; a power-invariant variant uses $\sqrt{2/3}$ – check the drive manual. Inverse Park uses $R(+\theta_e)$ [Park, 1929].

The same current seen in the three frames



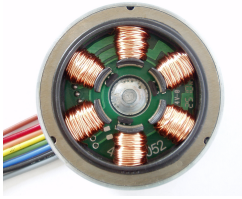
- Torque command steps from 5 A to 10 A at $\theta_e = 360°$.
- abc: three sines.
- $\alpha\beta$: two sines, same frequency.
- dq: **flat lines** – $i_d = 0$, i_q makes a clean step.
- A PI controller (W1D3) handles constants perfectly ⇒ no error at any speed.

Electrical vs. mechanical angle: $\theta_e = p\,\theta_m$



$$\theta_e = p\,\theta_m, \qquad f_e = \frac{p\,n}{60}$$

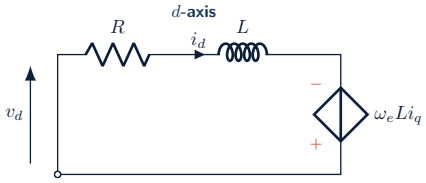
Example: $p = 4$, $n = 3000$ rpm:
$f_e = 4 \cdot 3000/60 = 200$ Hz, $\omega_e = 2\pi \cdot 200 = 1257$ rad/s.

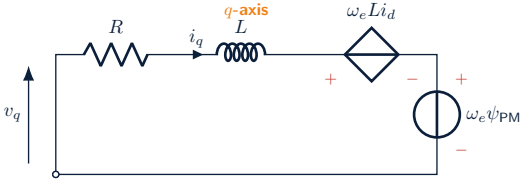


12-slot stator. Medvedev, CC BY-SA 3.0, via Wikimedia Commons

- $p =$ number of **pole pairs** (8 magnet poles $\Rightarrow p = 4$).
- One mechanical turn $= p$ electrical turns: the current must go through p full sine periods.
- An angle error of 2° mechanical $= 8^\circ$ electrical.

The motor seen by the rotating camera: two R–L circuits





$$v_d = R\,i_d + L\frac{di_d}{dt} - \omega_e L\,i_q$$

$$v_q = R\,i_q + L\frac{di_q}{dt} + \omega_e L\,i_d + \underbrace{\omega_e \psi_{\text{PM}}}_{\text{back-EMF}}$$

- Each axis = the W1D1 winding (R, L) plus speed-dependent sources; time constant $L/R = 6$ ms.
- Back-EMF appears only on q. The $\omega_e L\,i$ terms couple the axes.

Going further: derived from $\vec{v} = R\vec{i} + d\vec{\psi}/dt$ in rotating coordinates, which adds $j\omega_e\vec{\psi}$; IPM motors have $L_d \neq L_q$.

Torque comes from i_q

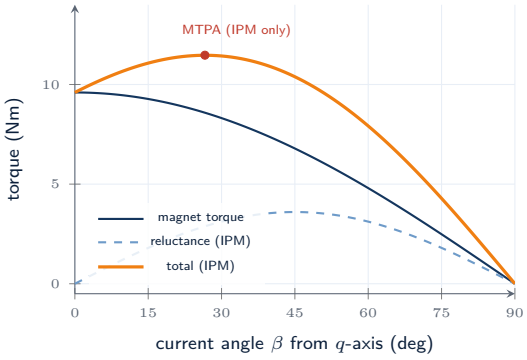
$$T_e = \tfrac{3}{2}\, p\, \psi_{\mathsf{PM}}\, i_q = k_t\, i_q$$

Example (reference motor)

$k_t = 1.5 \cdot 4 \cdot 0.12\,\mathrm{Vs} = 0.72\,\mathrm{Nm/A}$
$i_q = 10\,\mathrm{A} \Rightarrow T_e = 0.72 \cdot 10 = 7.2\,\mathrm{Nm}.$

- Set $i_d^* = 0$: all the current makes torque.
- If the current arrow is tilted by β away from q, only $\cos\beta$ of it is useful (blue curve).

Going further: in **IPM** motors ($L_q > L_d$) a negative i_d adds *reluctance torque* $\frac{3}{2}p(L_d - L_q)i_d i_q$; the best angle is **MTPA** (max. torque per amp, here $\beta = 26.6°$, +19 %). Above base speed, negative i_d weakens the magnet flux (**field weakening**) to reach higher speed [Jahns et al., 1986].



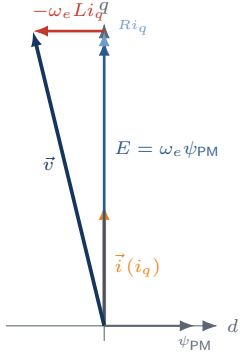
Illustrative IPM motor, $\hat{I} = 20\,\mathrm{A}$. For our surface-magnet motor only the blue curve exists: best at $\beta = 0$.

Worked example: operating point at 3000 rpm

Reference motor, $i_d = 0$, $i_q = 10\,\text{A}$, $n = 3000\,\text{rpm}$.

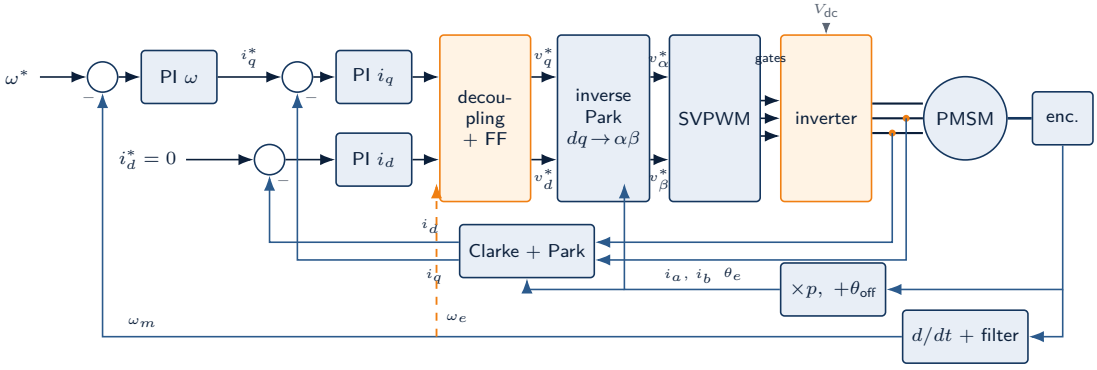
1. Electrical speed:
   $\omega_e = p \cdot \frac{2\pi n}{60} = 4 \cdot 314.2\,\text{rad/s} = 1257\,\text{rad/s}$.
2. Back-EMF: $E = \omega_e \psi_{\text{PM}} = 1257 \cdot 0.12 = 150.8\,\text{V}$.
3. Axis voltages:
   $v_q = R\,i_q + E = 5.0 + 150.8 = 155.8\,\text{V}$;
   $v_d = -\omega_e L\,i_q = -1257 \cdot 0.003 \cdot 10 = -37.7\,\text{V}$.
4. Length: $|\vec{v}| = \sqrt{37.7^2 + 155.8^2} = 160.3\,\text{V}$
   $< V_{\text{max}} = 188\,\text{V}$ ✓ (85 % used).

Torque $k_t i_q = 7.2\,\text{Nm}$; power $T\omega_m = 7.2 \cdot 314 = 2.26\,\text{kW}$.



Vector diagram to scale (40 V/cm). At higher speed or load the voltage arrow reaches the limit circle.

FOC: the basic structure



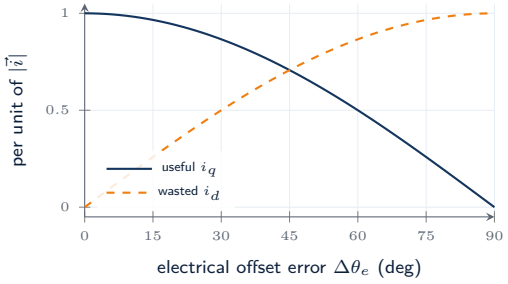
Every PWM period (62.5 µs): 1. measure i_a, i_b, angle; 2. Clarke + Park; 3. two PIs; 4. inverse Park; 5. new duty cycles.

Going further: decoupling: add the known terms $-\omega_e L i_q$ and $+\omega_e(L i_d + \psi_{\text{PM}})$ to the PI outputs, so each PI sees only its own R–L circuit.

Rotor angle alignment and offset errors

- The encoder's zero is wherever it was mounted; FOC needs $\theta_e = 0$ exactly at the magnet's d-axis.
- Wrong offset = the camera is mounted crooked: we push at the wrong angle.
- Fixes: absolute encoder with factory offset (typical); DC alignment (push a current, rotor snaps to $\theta_e = 0$); Hall sensors at start-up.

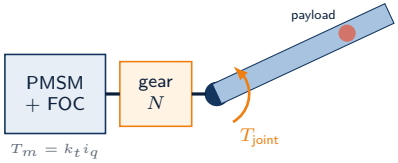
$$T_e = k_t \, |\vec{i}| \cos \Delta\theta_e$$



Example

2° mech., $p = 4 \Rightarrow 8^\circ$ el.: torque $-1\,\%$, but $14\,\%$ of the current wasted as heat. Beyond 90° the torque reverses $\Rightarrow$ **runaway**.

FOC in a robot joint: from joint torque to i_q





Strain-wave (harmonic) gear set. Pieceofmetalwork, CC BY-SA 4.0, via Wikimedia Commons

$$i_q^* = \frac{T_{\text{joint}}}{N\,\eta\,k_t}$$

Worked example (reference joint)

1. Joint needs $T_{\text{joint}} = 120$ Nm; gear $N = 100$, efficiency $\eta = 0.8$.
2. Motor torque: $120/(100 \cdot 0.8) = 1.5$ Nm.
3. Current: $i_q^* = 1.5/0.72 = 2.08$ A, $i_d^* = 0$.
4. Joint at 3 rad/s $\Rightarrow$ motor at 300 rad/s $\approx$ 2865 rpm – near the 3000 rpm example.

Take-away: the robot controller thinks in joint torque; FOC turns it into i_q.

Check your understanding – 3.3

Check your understanding

1. $i_a = 8\,\text{A}$, $i_b = -4\,\text{A}$, $i_c = -4\,\text{A}$. What are i_α and i_β?
2. In the "rotating camera" view, why do the motor currents look constant?
3. An 8-pole motor turns at 1500 rpm. What is the electrical frequency of its currents?

Discuss with your neighbour for 2 minutes.

Summary 3.3 – Field-oriented control

Key take-aways

- Three phase currents = one rotating arrow.
- Clarke: $abc \to \alpha\beta$; Park: view from the rotor $\Rightarrow$ DC values i_d, i_q.
- Torque $T_e = k_t i_q$; keep $i_d = 0$ (surface magnets).
- Each axis is an R–L circuit; back-EMF on q.
- FOC: measure, transform, two PIs, transform back, PWM – every period. Needs a correct angle.

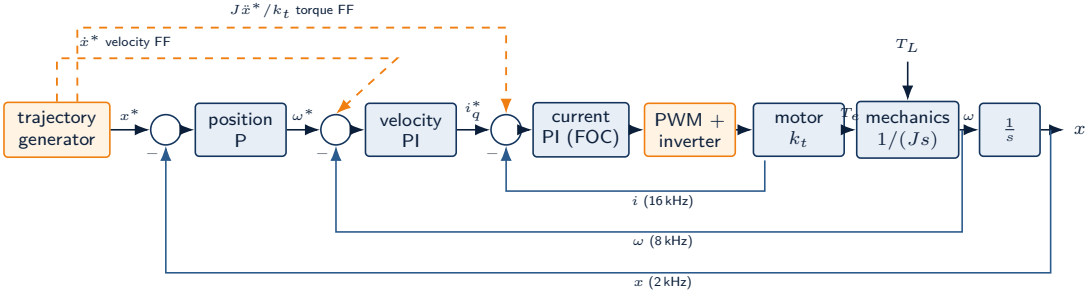
Key terms

PMSM – permanent-magnet synchronous motor.
$\alpha\beta$ **frame** – two fixed axes.
dq **frame** – two axes turning with the rotor.
i_d **/** i_q – flux / torque current.
Pole pairs p – $\theta_e = p\theta_m$.
MTPA, field weakening – see "going further".

Session 2

3.4 Cascaded Current, Velocity and Position Loops

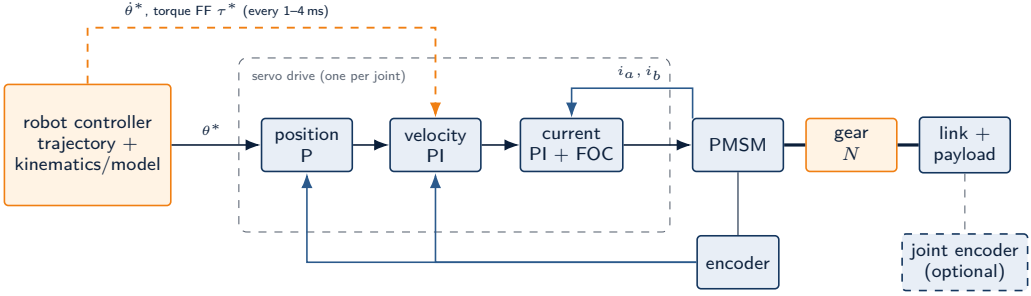
The nested (cascade) control structure



- Analogy: navigator (position) → cruise control (speed) → engine management (torque).
- Each loop treats the closed inner loop as its "actuator".
- Inner loops are **faster**; they also enforce limits (current → torque).
- Dashed: **feedforward** – known commands sent straight to the inner loops.

[Ellis, 2012]

Cascaded loops in a robot joint: who closes what?



- Most industrial arms: the drive closes all three loops; the robot controller sends set-points over a fieldbus (EtherCAT, W2D5).
- Computed-torque / force control (2.9): the controller sends **torque**; the drive runs only the current loop.
- Encoder on the motor sees motor angle; the link angle differs by gear **backlash** and **compliance** (W1D2).
- Cobots often add a joint-side encoder or torque sensor.

Current-loop PI: a step-by-step recipe

- Plant (one axis): the R–L winding, $G_i(s) = \dfrac{1}{R + Ls}$, plus a small **delay** T_σ (compute + PWM).
- PI: $C_i(s) = K_p \dfrac{1 + sT_i}{sT_i}$.
- Idea: let the PI's zero cancel the slow winding pole, then pick K_p so the delay is still safe.

$$T_i = \frac{L}{R}, \qquad K_p = \frac{L}{2\,T_\sigma}$$

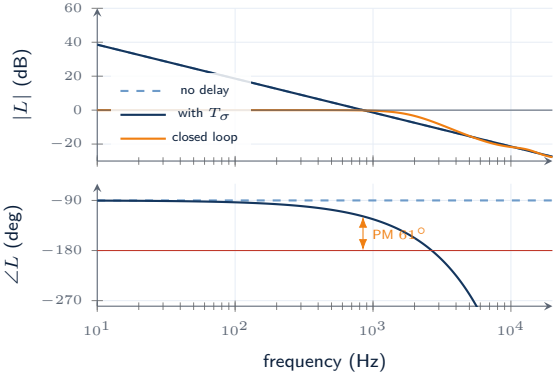
This rule is called the **technical optimum** (TO).

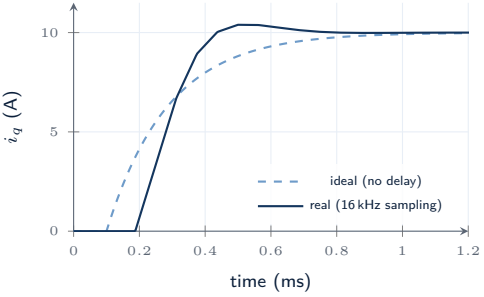
Going further: the closed loop becomes 2nd order with $\zeta = 0.707$ (4.3 % overshoot); saturation lowers L at high current, so check K_p there.

Worked example (reference motor)

1. $T_s = 1/16\,\text{kHz} = 62.5\,\mu\text{s}$; $T_\sigma = 1.5\,T_s = 93.75\,\mu\text{s}$.
2. $T_i = 3\,\text{mH}/0.5\,\Omega = 6\,\text{ms}$.
3. $K_p = \dfrac{3\,\text{mH}}{2 \cdot 93.75\,\mu\text{s}} = 16\,\text{V/A}$.
4. Crossover $\omega_c \approx K_p/L = 5333\,\text{rad/s}$ ($f_c \approx 850\,\text{Hz}$); delay costs $360^\circ \cdot 850 \cdot 93.75\,\mu\text{s} = 29^\circ \Rightarrow \text{PM} \approx 61^\circ$.

Current loop: computed frequency and step response





Step $0 \rightarrow 10$ A: current follows in ≈ 0.3 ms, overshoot only 3.9 %.
The delay bends the phase curve down at high frequency – that is what limits K_p.

Velocity-loop PI: a step-by-step recipe

- Plant: current $\rightarrow$ torque $\rightarrow$ speed. Newton: $J\dot{\omega} = k_t i_q \Rightarrow$ an **integrator** $\dfrac{k_t}{Js}$.
- Plus a small lag T_{eq}: closed current loop + speed filter.
- Choose a spacing factor a ($2 \ldots 4$): crossover a times below the lag, PI corner a times below the crossover.

$$K_{p\omega} = \frac{J\,\omega_c}{k_t}, \qquad \omega_c = \frac{1}{a\,T_{\text{eq}}}$$

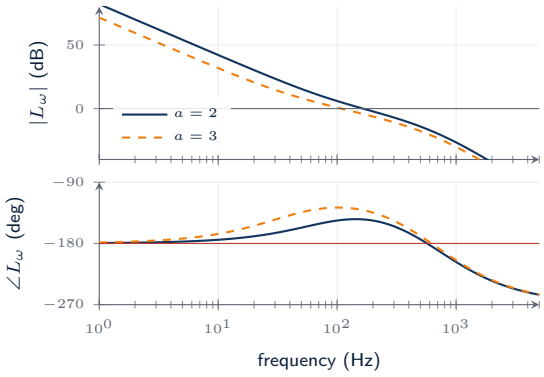
Going further: this is the **symmetric optimum** [Kessler, 1958]: $T_{i\omega} = a^2 T_{\text{eq}}$, PM $= \arctan\frac{a^2-1}{2a}$ (37° at $a=2$, 53° at $a=3$); a reference prefilter removes the large step overshoot.

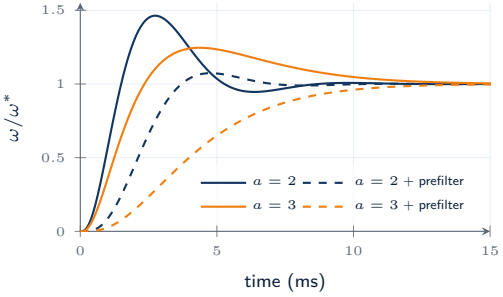
Worked example: spacing factor 2

1. $T_{\text{eq}} = 2 \cdot 93.75\,\mu\text{s} + 0.31\,\text{ms} \approx 0.5\,\text{ms}$.
2. $\omega_c = 1/(2 \cdot 0.5\,\text{ms}) = 1000\,\text{rad/s}$ (159 Hz).
3. $J = J_m(1+3) = 1.2 \cdot 10^{-3}\,\text{kg m}^2$; $K_{p\omega} = 1.2 \cdot 10^{-3} \cdot 1000/0.72 = 1.67\,\text{A s/rad}$.
4. $T_{i\omega} = a^2 T_{\text{eq}} = 2\,\text{ms}$.

Larger $a \Rightarrow$ slower but calmer ($a=3$: $f_c = 106\,\text{Hz}$).

Velocity loop: computed Bode plot and step responses



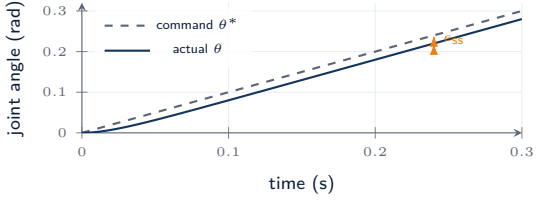


$a = 2$: 164 Hz, PM $35°$, 46 % overshoot (7 % with prefilter).
$a = 3$: 107 Hz, PM $53°$, 25 % (0 % with prefilter).
More phase margin = less overshoot.

Position loop: a P controller and the following error

- With a good velocity loop, speed $\approx$ command; position is the integral of speed.
- So a simple **P controller** reaches any fixed target with zero error. Gain k_v (1/s).
- Following a ramp (constant speed ω) leaves a constant lag: the **following error**.

$$e_{\text{ss}} = \frac{\omega}{k_v} \qquad \text{(P loop, no feedforward)}$$



Worked example: robot joint

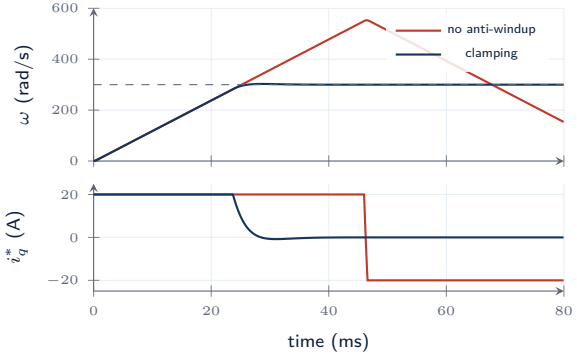
1. Joint loop $k_v = 50\,\text{s}^{-1}$, joint speed $\omega = 1\,\text{rad/s}$.
2. $e_{\text{ss}} = 1/50 = 0.02\,\text{rad} = 1.15^\circ$.
3. At 1 m reach the tool lags $0.02 \cdot 1 = 20\,\text{mm}$ (!)
4. Doubling k_v halves it – but stability limits k_v.

Analogy: a dog on a leash – the faster the walk, the longer the leash.

Six joints with **different** lags bend a straight tool path – feedforward (3.6) fixes this.

Torque and speed limits; integrator windup

- Limits: $|i_q^*| \le \hat{I}_{\max}$, $|\omega^*| \le \omega_{\max}$, plus the voltage limit.
- While the output is limited, the loop is effectively **open**; the integrator keeps adding up error.
- Result: **windup** – huge overshoot when the limit releases.
- Analogy: holding the accelerator floored and still "remembering" to push harder.
- Cure (**anti-windup**): stop integrating while saturated (clamping).



300 rad/s step, current limit 20 A: peak 553 rad/s vs. 303 rad/s.

Check your understanding – 3.4

Check your understanding

1. $L = 6\,\text{mH}$, $T_\sigma = 100\,\mu\text{s}$. What current-loop gain K_p does the recipe give?
2. A robot joint turns at $0.5\,\text{rad/s}$ with $k_v = 25\,\text{s}^{-1}$ and no feedforward. Following error in degrees?
3. Why must the loops be tuned from the inside out?

Discuss with your neighbour for 2 minutes.

Summary 3.4 – Cascaded loops

Key take-aways

- Cascade: position (P) $\rightarrow$ velocity (PI) $\rightarrow$ current (PI); inner loops are faster.
- Current PI: $T_i = L/R$, $K_p = L/(2T_\sigma) \Rightarrow$ about 850 Hz.
- Velocity PI: $K_p = J\omega_c/k_t$ with $\omega_c = 1/(aT_{\text{eq}})$.
- P position loop: following error ω/k_v.
- Limits need anti-windup; in robots the drive or the controller may close the outer loops.

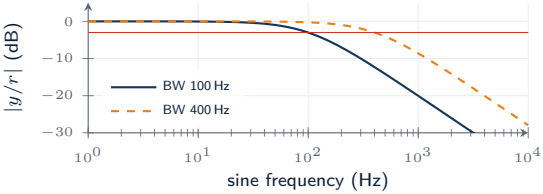
Key terms

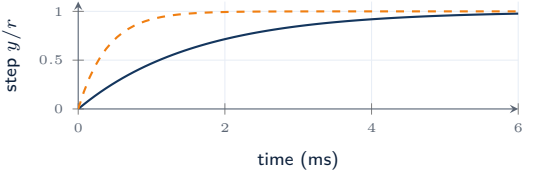
Cascade – loops nested inside each other.
Crossover ω_c – where loop gain is 1.
k_v – position-loop gain (1/s).
Following error – lag behind a moving target.
Windup – integrator overfills during a limit.
Technical / symmetric optimum – PI recipes.

Session 3

3.5 Loop Tuning and Bandwidth

Refresher: what does “bandwidth” mean?





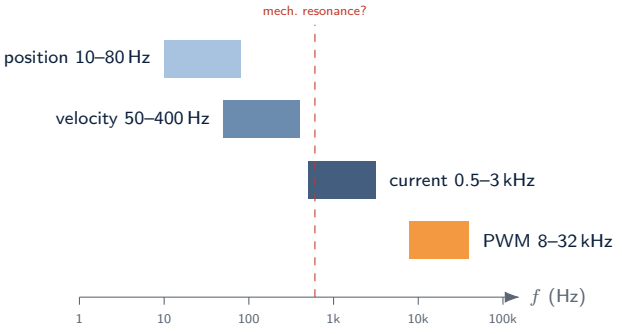
- Feed the closed loop a sine command and watch the output.
- Slow sines: followed perfectly. Fast sines: output too small and late.
- **Bandwidth** = frequency where the output drops to $70\,\%$ ($-3\,\mathrm{dB}$).
- Analogy: a dancer copying a leader – slow moves are easy, fast ones get lost.

$$\text{rise time } t_r \approx \frac{0.35}{f_{\mathrm{BW}}}$$

Example: $f_{\mathrm{BW}} = 100\,\mathrm{Hz} \Rightarrow t_r \approx 3.5\,\mathrm{ms}$; $400\,\mathrm{Hz} \Rightarrow 0.9\,\mathrm{ms}$.

Take-away: higher bandwidth = faster response.

The bandwidth hierarchy of a servo drive



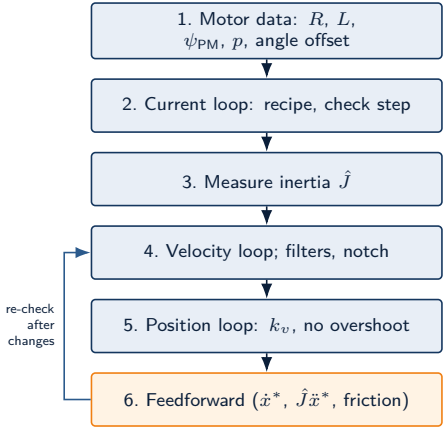
- Each loop is a few times slower than the one inside it.
- Current: limited by sampling and PWM delay. Velocity: by **mechanical resonances** and filters. Position: by the velocity loop.



Industrial robot. Oleg Yunakov, CC BY-SA 4.0, via Wikimedia Commons

Robots with elastic gears: velocity bandwidth only a few tens of Hz; stiff direct drives: several hundred Hz [Ellis, 2012].

Tuning procedure: from the inside out



- Each outer loop is designed on the **closed** inner loop – tune outer first and you tune against the wrong plant.
- Measure, don't guess: e.g. inertia from an acceleration test, $\hat{J} = k_t\,\Delta i_q / \Delta\dot{\omega}$.

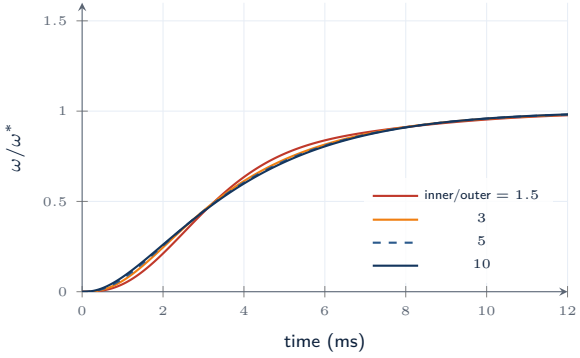
Example

$\Delta i_q = 5\,\text{A}$ gives $\Delta\dot{\omega} = 3000\,\text{rad/s}^2$:
$\hat{J} = 0.72 \cdot 5/3000 = 1.2 \times 10^{-3}\,\text{kg m}^2$.

- Raise gains until a margin limit (PM $\geq 45°$, GM $\geq 6\,\text{dB}$), then back off.
- Check large steps last (limits, anti-windup).

[Ellis, 2012]; [Åström & Hägglund, 2006]

Bandwidth separation: rule of thumb 3–10×



Computed: velocity PI designed for 1000 rad/s *assuming a perfect current loop*; the real current loop has bandwidth ω_i.

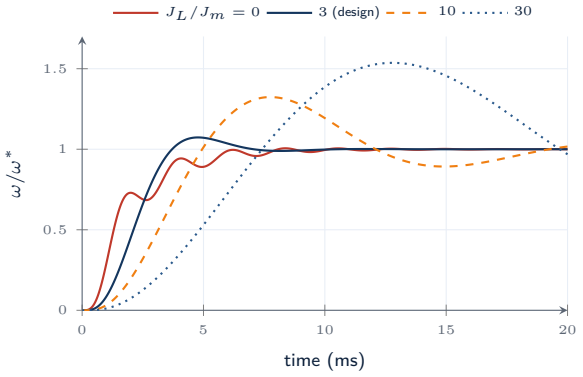
- A slow inner loop adds lag (phase loss) at the outer crossover:

$$\Delta\varphi = \arctan(\omega_c/\omega_i)$$

ω_i/ω_c	phase lost
1.5	33.7°
3	18.4°
10	5.7°

- Below 3×: ringing (red curve).
- Above 10×: safe, but the outer loop could be faster.

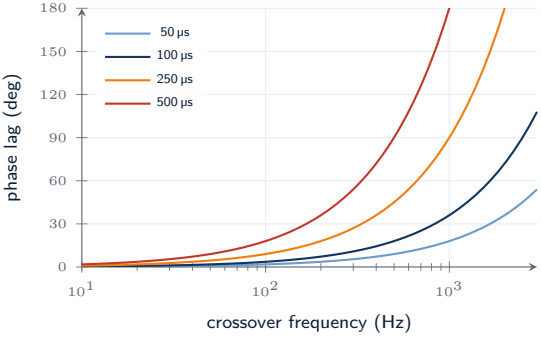
Effect of the load inertia ratio



Same gains (designed for ratio 3). Crossover / PM: 0: 439 Hz/12°; 3: 164 Hz/35°; 10: 80 Hz/31°; 30: 43 Hz/21°.

- Loop gain $\propto K_{p\omega} k_t / J$: more load inertia $\Rightarrow$ **lower crossover**, sluggish; less $\Rightarrow$ higher, towards the delays, oscillating.
- Robot arm: the inertia seen by joint 1 changes with the **pose** (arm stretched vs. folded), divided by N^2 through the gear (W1D2).
- Rule: tune for the worst pose, or schedule gains with pose; aim for $J_L/J_m \leq 3 \ldots 10$.

Delays and their phase cost at crossover



Analogy: a shower with a long pipe – you turn the tap, the temperature changes seconds later, so you overshoot.

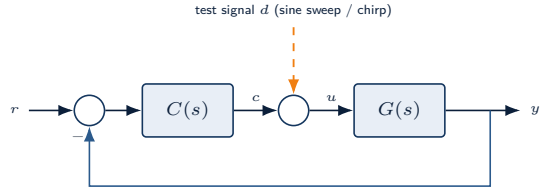
$$\Delta\varphi = 360^\circ \cdot f_c \cdot T_d$$

Delay source	typical
Computation (1 sample at 16 kHz)	62.5 µs
PWM update ($T_s/2$)	31 µs
Encoder data transfer	10–30 µs
Velocity filter	0.1–1 ms

Example

$T_d = 0.5$ ms, $f_c = 160$ Hz: $360 \cdot 160 \cdot 0.0005 = 28.8^\circ$ of phase margin lost.

Measuring frequency responses (FRFs)



plant $G(j\omega) = \dfrac{Y(j\omega)}{U(j\omega)}$ at each test frequency

- Add a small test signal *inside* the running loop – the machine stays under control.
- Compare output and input amplitude and phase at each frequency ⇒ a measured Bode plot.
- Large enough to beat friction and noise, small enough to stay linear.

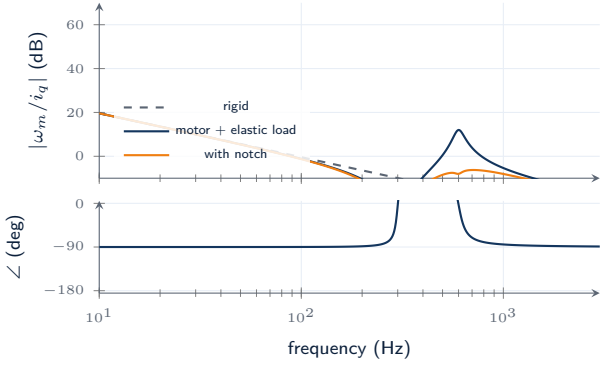


HP 3562A dynamic signal analyzer. Loxyger, CC BY 4.0, via Wikimedia Commons

Modern drives have this analyser built in: the software injects a chirp and plots the Bode diagram.

Going further: estimate $\hat{G} = \hat{S}_{uy}/\hat{S}_{uu}$ from cross-spectra and check the **coherence** $\gamma^2 \approx 1$.

A measured-style FRF: resonance and notch filter



- Motor and load joined by a springy shaft = **two-mass system**.
- Analogy: a playground swing – pushed at its own frequency, it swings huge.
- Dip (anti-resonance) at 300 Hz, peak (**resonance**) at 600 Hz.
- The peak can push the loop gain above 1 where the phase is bad ⇒ "buzz".
- **Notch filter**: cuts a narrow band around 600 Hz in the current command.

Going further: $f_a = \frac{1}{2\pi}\sqrt{k_s/J_L}$,
$f_r = f_a\sqrt{1 + J_L/J_m}$; notch
$N(s) = \frac{s^2+2\zeta_z\omega_r s+\omega_r^2}{s^2+2\zeta_p\omega_r s+\omega_r^2}$. More in W2D1, session 3.7.

Check your understanding – 3.5

Check your understanding

1. A velocity loop has a bandwidth of 200 Hz. Roughly what rise time do you expect?
2. How much phase does a 200 µs delay cost at a 250 Hz crossover?
3. The load inertia doubles but the gains stay the same. What happens to the crossover frequency?

Discuss with your neighbour for 2 minutes.

Summary 3.5 – Tuning and bandwidth

Key take-aways

- Bandwidth = how fast a loop can follow; $t_r \approx 0.35/f_{\mathrm{BW}}$.
- Tune inside out; separate bandwidths 3–10×.
- Inertia changes the loop gain: retune with $\hat{J}$.
- Every delay costs $360^\circ f_c T_d$ of phase.
- Measure FRFs in closed loop; tame resonances with notches.

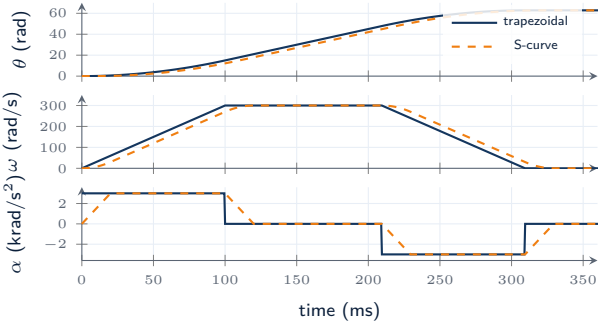
Key terms

Bandwidth – -3 dB frequency.
Phase margin – safety from oscillation.
Inertia ratio – J_L/J_m.
FRF – measured frequency response.
Resonance – springy mechanics amplifying one frequency.
Notch filter – removes a narrow band.

Session 4

3.6 Trajectory Generation, Feedforward and Disturbance Compensation

Trajectory generation: trapezoidal vs. S-curve profiles

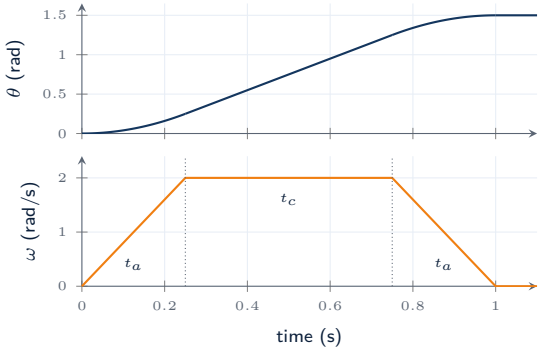


Move 10 revolutions, max. 300 rad/s, max. 3000 rad/s^2.

- **Trapezoidal**: speed ramps; acceleration *jumps* $\Rightarrow$ jolts that shake the machine. Total 309 ms.
- **S-curve**: acceleration ramps too (limited **jerk**). Total 329 ms (+6 %), much smoother.
- Analogy: a good lift driver – you never feel the start.
- Peak torque $J\alpha = 3.6\,\text{Nm}$ ($i_q = 5\,\text{A}$) – within limits.

[Biagiotti & Melchiorri, 2008]

Worked example: planning a trapezoidal joint move



Move a joint by $\Delta\theta = 1.5\,\text{rad}$ (86°) with $\omega_{\max} = 2\,\text{rad/s}$, $\alpha_{\max} = 8\,\text{rad/s}^2$.

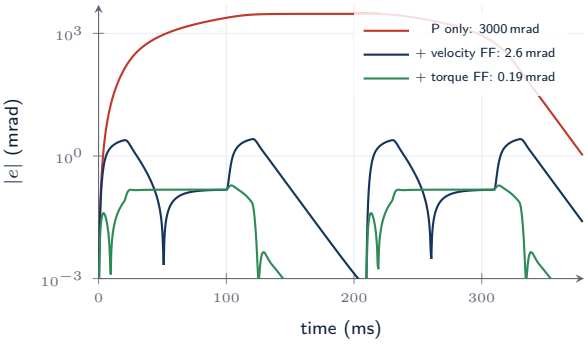
Step by step

1. Ramp time $t_a = \omega_{\max}/\alpha_{\max} = 0.25\,\text{s}$.
2. Angle per ramp $\frac{1}{2}\alpha t_a^2 = 0.25\,\text{rad}$; both ramps $0.5\,\text{rad}$.
3. Cruise: $1.0\,\text{rad}/2\,\text{rad/s} = 0.5\,\text{s}$.
4. Total $T = 2t_a + t_c = 1.0\,\text{s}$.

cruise exists only if $\Delta\theta \geq \omega_{\max}^2/\alpha_{\max}$ (here $0.5\,\text{rad}$)

Otherwise the profile is a **triangle**. Robot controllers plan all joints to finish **together** (synchronised PTP, W2D2).

Following error and how feedforward removes it



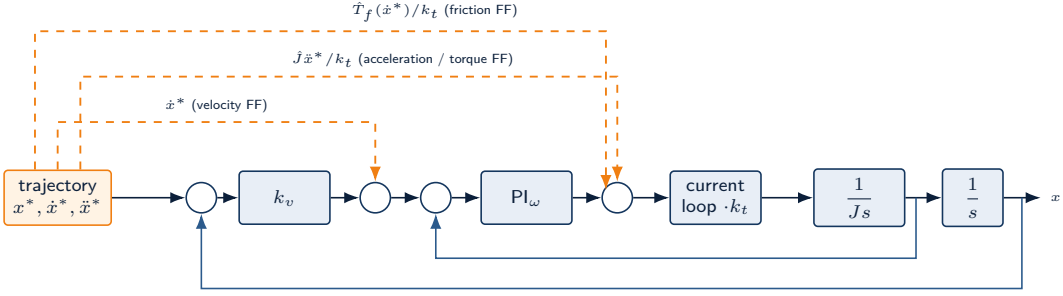
Simulated cascade on the S-curve move; legend: peak $|e|$. **Log scale**: each step is $\sim 1000\times$.



5-axis machining centre. G. McKechnie, CC BY-SA 2.0, via Wikimedia Commons

- Velocity FF: send the planned speed $\dot{x}^*$ directly.
- Torque FF: send the needed current $\hat{J}\ddot{x}^*/k_t$ directly.
- Feedback only fixes what is left. FF acts outside the loop: **no stability change**.

Feedforward: velocity, acceleration/torque and friction

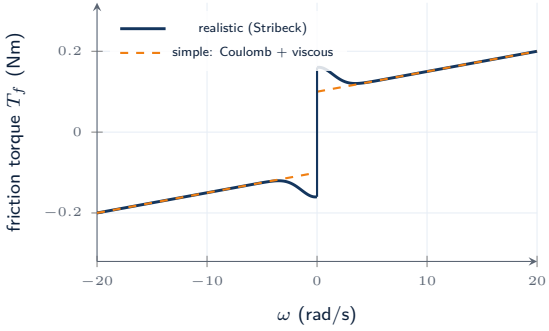


$$i_{q,\mathrm{FF}}^* = \left(\hat{J}\ddot{x}^* + \hat{T}_f\right)/k_t$$

Example: $\hat{J} = 1.2 \times 10^{-3}\,\mathrm{kg\,m^2}$, $\ddot{x}^* = 3000\,\mathrm{rad/s^2}$, $\hat{T}_f = 0.1\,\mathrm{Nm}$: $(3.6 + 0.1)/0.72 = 5.1\,\mathrm{A}$.

- Feedforward = **predict** the needed current from the plan; feedback only corrects the leftover.
- Needs smooth (S-curve) commands and a good $\hat{J}$.
- Outside the loop: no stability change.

Disturbances: load torque and friction



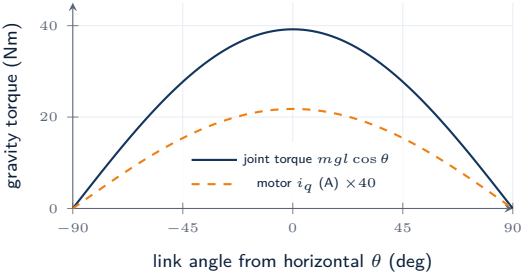
Breakaway 0.16 Nm, sliding 0.1 Nm, plus 5 mNm per rad/s.

$$J\dot{\omega} = k_t i_q - T_L - T_f(\omega)$$

- **Load torque** T_L: cutting forces, gravity on vertical axes.
- **Friction**: jumps at $\omega = 0$ (harder to start than to keep sliding) $\Rightarrow$ stick-slip, small jerks when reversing.
- The PI removes a constant T_L eventually, but a **dip** occurs first.
- Robots: also **gravity** on every link (next slide).

[Armstrong-Hélouvry, Dupont & Canudas de Wit, 1994]

Gravity compensation for a robot joint





Robots in a car-body welding line. Mixabest, CC BY-SA 3.0, via Wikimedia Commons

- Gravity pulls every link down: a load torque that **changes with pose**.
- Pure PI: the integrator must "discover" it – sag and overshoot after each move.
- Better: compute it from the model and add it as feedforward (2.9).

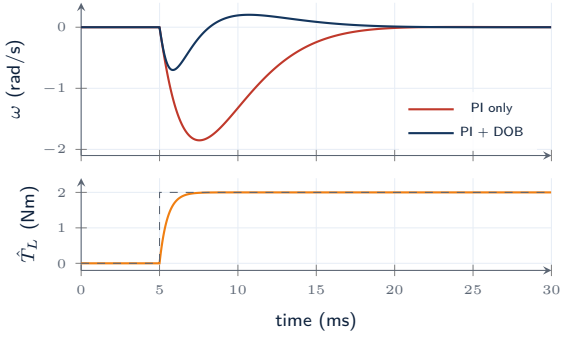
$$\tau_g = m\,g\,l\cos\theta, \qquad i_{q,\mathrm{g}} = \frac{\tau_g}{N\,k_t}$$

Example

$m = 8\,\mathrm{kg}$, $l = 0.5\,\mathrm{m}$: $\tau_g = 8 \cdot 9.81 \cdot 0.5 = 39.2\,\mathrm{Nm}$ (horizontal). With $N = 100$: motor $0.392\,\mathrm{Nm}$, $i_q = 0.54\,\mathrm{A}$ just to hold still.

Going further: for several links the gravity vector $\boldsymbol{g}(\boldsymbol{\theta})$ comes from the robot's dynamic model (W2D2, 4.6).

Disturbance observer (DOB): estimate the unknown push



Simulated: holding $\omega^* = 0$, load step 2 Nm at 5 ms.

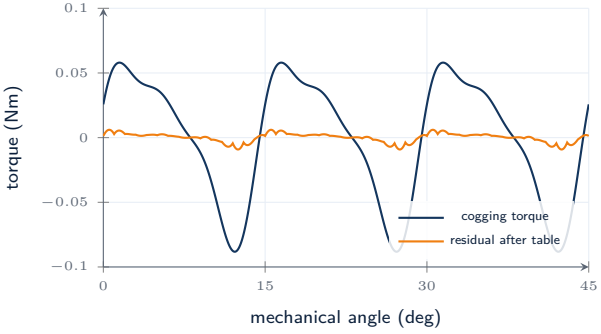
- Idea: compare what the motor *should* do ($k_t i_q$) with what it *does* ($\hat{J}\dot{\omega}$). The difference is the unknown disturbance.

$$\hat{T}_L \approx k_t\, i_q - \hat{J}\,\dot{\omega} \quad \text{(low-pass filtered)}$$

- Add $\hat{T}_L / k_t$ to the current command $\Rightarrow$ the disturbance is cancelled.
- Result: speed dip 1.85 rad/s $\rightarrow$ 0.70 rad/s (2.6$\times$ smaller).
- Bonus: $\hat{T}_L$ detects **collisions** – used in cobots to stop on contact.

Going further: the filter $Q(s) = 1/(1 + s\tau_Q)$ makes the derivative usable; $1/\tau_Q \approx 0.5$–$2\times\omega_c$, limited by noise and delay [Ohnishi, Shibata & Murakami, 1996].

Cogging and torque ripple compensation



Illustrative 12-slot/8-pole motor: 0.146 Nm → 0.015 Nm peak-to-peak.

- **Cogging**: magnets "click" into the slots – felt when you turn an unpowered motor by hand.
- Repeats every $360^\circ/\mathrm{LCM}(\text{slots}, \text{poles})$.

Example

12 slots, 8 poles: LCM $= 24 \Rightarrow$ period $360^\circ/24 = 15^\circ$ mech.

- Cure: measure i_q vs. angle at slow speed, store in a **look-up table**, add it to i_q^*.
- Matters most at low speed.

Check your understanding – 3.6

Check your understanding

1. A joint must move 0.3 rad with $\omega_{\max} = 2\,\mathrm{rad/s}$ and $\alpha_{\max} = 8\,\mathrm{rad/s^2}$. Trapezoid or triangle?
2. A robot arm holds a load horizontally. Which compensation would you add, and why does it depend on the pose?
3. Explain in one sentence how a disturbance observer "sees" a collision.

Discuss with your neighbour for 2 minutes.

Summary 3.6 – Trajectories, feedforward, disturbances

Key take-aways

- Trajectory generator gives θ^*, $\dot{\theta}^*$, $\ddot{\theta}^*$; S-curves limit jerk.
- Feedforward sends the *known* speed and torque directly: following error almost vanishes.
- Gravity, friction and load push against the joint; model what you can.
- A DOB estimates the *unknown* push from $k_t i_q - \hat{J}\dot{\omega}$.
- Cogging repeats with angle: table compensation.

Key terms

Trapezoidal profile – ramp, cruise, ramp.
Jerk – rate of change of acceleration.
Feedforward – act on the known command.
Gravity compensation – $mgl\cos\theta$ added to i_q^*.
DOB – disturbance observer.
Cogging – magnet–slot torque at zero current.

Day summary

From voltage to joint torque

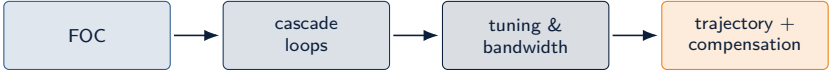
3.3 FOC: Clarke + Park (rotating camera) turn AC into DC; torque $k_t i_q$; joint torque $N k_t i_q$.

3.4 Cascade P–PI–PI with simple recipes; following error ω / k_v; anti-windup.

From loops to accurate robot moves

3.5 Bandwidth, inside-out tuning, 3–10× separation, pose-dependent inertia, delay cost.

3.6 Trapezoid/S-curve moves, feedforward, gravity compensation, observers.



Week 1 done: actuators → control → servo drives. Week 2 puts them into a robot arm.

Preview of Week 2 Day 1: Servo III + Robot Mechanisms I

- **3.7** Resonance, compliance and notch filters in robot joints – today's FRF peak, explained.
- **4.1** Robot anatomy: joints, DOF, workspace; articulated, SCARA, gantry, delta, cobots, mobile robots.
- **4.2** Spatial descriptions: rotation matrices and homogeneous transforms.
- **4.3** Forward kinematics (DH convention; planar 2-link, SCARA).



Industrial robot. Oleg Yunakov, CC BY-SA 4.0, via Wikimedia Commons

Bring along from today
– the rotation matrix $R(\theta)$ (Park),
– the two-mass FRF and the notch,
– joint torque $= N\eta k_t i_q$.

Figure credits

All plots, block diagrams, vector diagrams and schematics without a credit line are original drawings computed for this course (data generated by `w1d5/src/gen_data.py`). Downloaded figures (Wikimedia Commons):

File	Source (Wikimedia Commons) and author	Licence
d4_servo_drives.png	File:Servo motor drive.png – Tero Kontkanen	CC BY-SA 3.0
d4_bldc_stator.jpg	File:Stator Winding of a BLDC Motor.jpg – Medvedev	CC BY-SA 3.0
d1_harmonic_drive.jpg	File:Harmonic Drive AG strain wave gear set.jpg – Pieceofmetalwork	CC BY-SA 4.0
d4_kuka_robot.jpg	File:KUKA industrial robotic arm performing "The Beach" by Roy Nachum - 02.jpg – Oleg Yunakov	CC BY-SA 4.0
d4_signal_analyzer.jpg	File:3562A Dynamic Signal Analyzer HP.jpg – Loxyger	CC BY 4.0
d4_machining_center.jpg	File:DeckelMaho-DMU50e-MachiningCenter.jpg – Glenn McKechnie	CC BY-SA 2.0
d1_bldc.jpg	File:Floppy drive spindle motor open.jpg – Sebastian Koppehel	CC BY 3.0
r15_robot_cell.jpg	File:KUKA Industrial Robots IR.jpg – Mixabest	CC BY-SA 3.0

References

- Krishnan, R.: *Permanent Magnet Synchronous and Brushless DC Motor Drives*. CRC Press, 2010.
- Leonhard, W.: *Control of Electrical Drives*, 3rd ed. Springer, 2001.
- Vas, P.: *Sensorless Vector and Direct Torque Control*. Oxford Univ. Press, 1998.
- Blaschke, F.: The principle of field orientation ... *Siemens Review* 39, 1972.
- Park, R. H.: Two-reaction theory of synchronous machines. *AIEE Trans.* 48, 1929.
- Clarke, E.: *Circuit Analysis of A-C Power Systems*. Wiley, 1943.
- Jahns, T.; Kliman, G.; Neumann, T.: Interior PM synchronous motors for adjustable-speed drives. *IEEE Trans. Ind. Appl.* 22(4), 1986.
- Kessler, C.: Das symmetrische Optimum. *Regelungstechnik* 6, 1958.
- Ellis, G.: *Control System Design Guide*, 4th ed. Butterworth-Heinemann, 2012.
- Åström, K. J.; Hägglund, T.: *Advanced PID Control*. ISA, 2006.
- Åström, K. J.; Murray, R. M.: *Feedback Systems*. Princeton Univ. Press, 2008.
- Spong, M. W.; Hutchinson, S.; Vidyasagar, M.: *Robot Modeling and Control*. Wiley, 2006.
- Siciliano, B.; Sciavicco, L.; Villani, L.; Oriolo, G.: *Robotics: Modelling, Planning and Control*. Springer, 2009.
- Craig, J. J.: *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson, 2005.
- Ohnishi, K.; Shibata, M.; Murakami, T.: Motion control for advanced mechatronics. *IEEE/ASME Trans. Mechatronics* 1(1), 1996.
- Armstrong-Hélouvry, B.; Dupont, P.; Canudas de Wit, C.: A survey of models, analysis tools and compensation methods for the control of machines with friction. *Automatica* 30(7), 1994.
- Biagiotti, L.; Melchiorri, C.: *Trajectory Planning for Automatic Machines and Robots*. Springer, 2008.
- Microchip AN1078: Sensorless field oriented control of a PMSM. IEC 61800-2: Adjustable speed electrical power drive systems.